

CSCE 465/765: Computer & Network Security

Homework 2

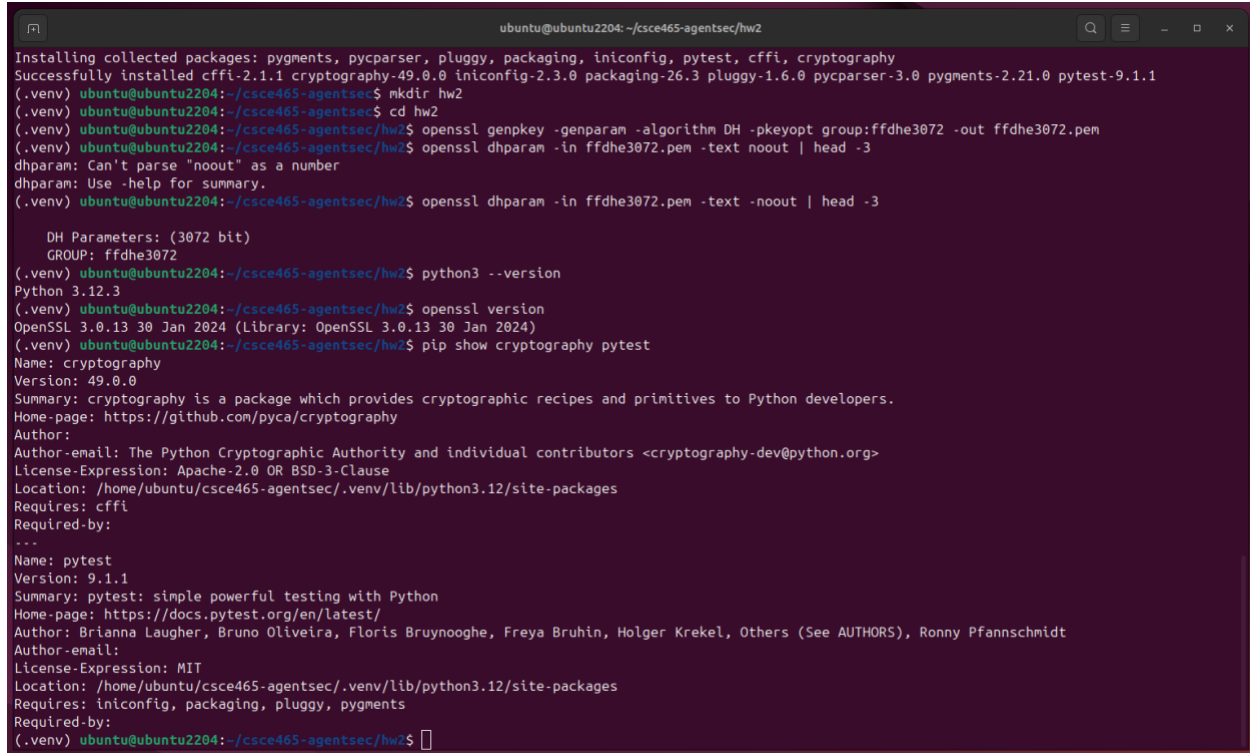
Protect Agent Messages with Classic Cryptography

Juel Sandy

832005296

October 4, 2026

Lab Preparation



```
ubuntu@ubuntu2204: ~/csce465-agentsec/hw2
Installing collected packages: pygments, pycparser, pluggy, packaging, iniconfig, pytest, cffi, cryptography
Successfully installed cffi-2.1.1 cryptography-49.0.0 iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0 pycparser-3.0 pygments-2.21.0 pytest-9.1.1
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ mkdir hw2
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ cd hw2
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl genpkey -genparam -algorithm DH -pkeyopt group:ffdhe3072 -out ffdhe3072.pem
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl dhparam -in ffdhe3072.pem -text noout | head -3
dhparam: Can't parse "noout" as a number
dhparam: Use -help for summary.
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
DH Parameters: (3072 bit)
GROUP: ffdhe3072
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

Figure 1. Python, OpenSSL, and python package versions

Task 1: Why Encryption Alone Is Insufficient

1.1 Design

For this task, I wanted to create a MITM type scenario so I created three parties on localhost. A sender, relay (port 8888), receiver (port 8889). I first encrypted a string with AES-256-CTR with no MAC and sent the ciphertext from the client. From the MITM, because I was aware of the plaintext, I knew what byte locations contained the text I wanted to modify. To determine how much I needed to XOR the value to obtain my desired text, I first got the mask between my original and desired value.

1.2 What Happened

```

baseline_ctr.py x secure_record.py
baseline_ctr.py > handle_mitm
19 def relay(ciphertext):
31     return bytes(ciphertext_bytearray)
32
33
34
35 def handle_mitm():
36     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
37     downstream = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
38
39     try:
40         server.bind((HOST, LOCAL_PORT))
41         server.listen(5)
42
43         client_socket, _ = server.accept()
44         downstream.connect((HOST, REMOTE_PORT))
45
46         ciphertext = client_socket.recv(4096)
47         modified = relay(ciphertext)
48
49         print("Intercepted client data")
50         print("Original: ", ciphertext.hex())
51         print("Modified: ", modified.hex())
52
53         # Forward toward downstream
54         downstream.sendall(modified)
55         time.sleep(5)
56         downstream.sendall(modified)
57
58     finally:
59         server.close()
60         downstream.close()
61
62 def deliver_message(source: socket):
63     try:
64         while True:

```

```

Problems Output Debug Console Terminal Ports
ubuntu@ubuntu2204:~/csc465-agentsec/hw2$ ./usr/bin/python3 /home/ubuntu/csc465-agentsec/hw2/baseline_ctr.py
Ciphertext upon dispatch: a2827d9b3bf4795e410bd13dc46dae6e4dcec9a2d7454b0bed70371da55a3b5d76bc2a6a9ecea6
Intercepted client data
[*] Accepted connection from ('127.0.0.1', 41808)
Original:  a2827d9b3bf4795e410bd13dc46dae6e4dcec9a2d7454b0bed70371da55a3b5d76bc2a6a9ecea6
Modified:  a2827d9b3bf4795e410bd13dd978aa644dcec9a2d7454b0bed70371da55a3b5d76bc2a6a9ecea6
[*] Received ciphertext from client at 02:56:49.653793
Decrypted: {"action": "OPEN", "path": "notes.txt"}
[*] Received ciphertext from client at 02:56:54.654159
Decrypted: {"action": "OPEN", "path": "notes.txt"}

```

Figure 2. Relay output: original vs. modified ciphertext, as well as a replay attack

When I XORed the ciphertext with the mask, the bits for the ciphertext and relevant plaintext values also flipped. From the relay, I then forwarded the modified ciphertext to the end destination where it was successfully decrypted. I was able to change the action READ to OPEN without learning the decryption key.

1.3 Root Cause

The root cause of this is that a flipped ciphertext bit using the CTR mode of operation, also causes the reciprocal plaintext bit to be flipped. Additionally, there was no MAC in place to authenticate that the ciphertext had not been modified. This also meant that I was able to replay the same ciphertext at any time and the client was unaware that it had been tampered with. This shows that confidentiality alone does not provide integrity or replay protection.

Task 2: Authenticated Finite-Field Diffie–Hellman

```
baseline_ctr.py x handshake.py x ffdhe3072.pem x secure_record.py •
handshake.py > main
137 def main():
138     # ...
159
160     signature = sign_transcript(gateway_th, TRANSCRIPT_ROLE_GATEWAY)
161     keys = generate_keys(gateway_credentials, node_pubk, gateway_th)
162     final_response, gateway_keys = {'role': 'Gateway', 'identity': TRANSCRIPT_ROLE_GATEWAY, 'signature': signature}, keys
163
164     # Node responds for final part of handshake
165     verify_message_role_and_iden(final_response, "Gateway")
166     accept_session(final_response["signature"], node_th, TRANSCRIPT_ROLE_GATEWAY, final_response["identity"], TRANSCRIPT_ROLE_GATEWAY)
167     node_keys = generate_keys(node_credentials, verify_public_key(node_credentials.public_key), node_th)
168
169     # Handshake complete, check if keys match
170     print("Handshake complete")
171     print(f"Gateway keys: {gateway_keys}")
172     print(f"Node keys: {node_keys}")
173     print(f"Keys match: {gateway_keys == node_keys}")
174
175 if __name__ == "__main__":
176     main()
177
Problems Output Debug Console Terminal Ports
ubuntu@ubuntu2204:~/csc465-agentsec/hw2$ /usr/bin/python3 /home/ubuntu/csc465-agentsec/hw2/handshake.py
Handshake complete
Gateway keys: {'K_g2n_enc': b'\xf7\xab\x1e\x8e\xd7\xaa\xcf\xcb\xdb\x99\x8e\x8d\xbd\x2f\x4d\x41\x10'\x11\x8a\xfa\xda\x1a\x6u', 'K_g2n_mac': b'\x82\xde\x1d\x1e\x5\x15\x1e\xfb\x1b\x1b\xdb\x3v\x9c\x91k' se7\x93j\x17j\x9as\x1d\x2e\x1e', 'K_n2g_enc': b'\x7f\xbb\x3e\x7f\x00\x9e\xee\x8f\xbb\x1a\x81\x92\x89\x18\x9e\x9b\x67\x1c2\x00\x15:\xcff\x9a9\x21\x3a\x3e4t\x1f6g', 'K_n2g_mac': b'\xdf\xda\x83f\x1c\x185\xed-\xf95+\x12\x9d9\xca9\xee7\x95\n\x9d\xcb\x1e\x04\x16a0\x13\x1b\x1b\x8d\x98p', 'session_id': b'\xaa1\x5N' @\xeb\x1c7\xeb\x1c7\x161y\x9d9\x1f0\x12.t\xfc\xdb-\x1b:\x16\x1c0\x18'}
Node keys: {'K_g2n_enc': b'\xf7\xab\x1e\x8e\xd7\xaa\xcf\xcb\xdb\x99\x8e\x8d\xbd\x2f\x4d\x41\x10'\x11\x8a\xfa\xda\x1a\x6u', 'K_g2n_mac': b'\x82\xde\x1d\x1e\x5\x15\x1e\xfb\x1b\x1b\xdb\x3v\x9c\x91k' se7\x93j\x17j\x9as\x1d\x2e\x1e', 'K_n2g_enc': b'\x7f\xbb\x3e\x7f\x00\x9e\xee\x8f\xbb\x1a\x81\x92\x89\x18\x9e\x9b\x67\x1c2\x00\x15:\xcff\x9a9\x21\x3a\x3e4t\x1f6g', 'K_n2g_mac': b'\xdf\xda\x83f\x1c\x185\xed-\xf95+\x12\x9d9\xca9\xee7\x95\n\x9d\xcb\x1e\x04\x16a0\x13\x1b\x1b\x8d\x98p', 'session_id': b'\xaa1\x5N' @\xeb\x1c7\xeb\x1c7\x161y\x9d9\x1f0\x12.t\xfc\xdb-\x1b:\x16\x1c0\x18'}
Keys match: True
ubuntu@ubuntu2204:~/csc465-agentsec/hw2$
```

Figure 3. Output of handshake.py

2.1 Connection to Textbook Concepts

When coding this handshake, it required knowledge of many topics from the textbook such as Diffie-Hellman key establishment, digital signatures & entity authentication, nonces, forward secrecy and key separation. I utilized Diffie-Hellman to eventually compute Z which was the shared secret between both parties, computed from their private and public keys.

Digital signatures & entity authentication was used in this project by utilizing RSA-PSS to bind each Diffie-Hellman share to a known identity. Fresh nonces were utilized each time to make every transcript unique which rendered old signatures from being reusable. Finally, I utilized the concept of forward secrecy by discarding the ephemeral Diffie Hellman private values. Even if an RSA key was leaked in the future, past sessions could not be exposed as the private values were discarded.

Distinct labels were given to each key when being hashed for the purposes of key separation. This concept allows us to limit the blast radius of a key being compromised as each key is not reused and has separate responsibilities.

2.2 Man-in-the-Middle and Reflection

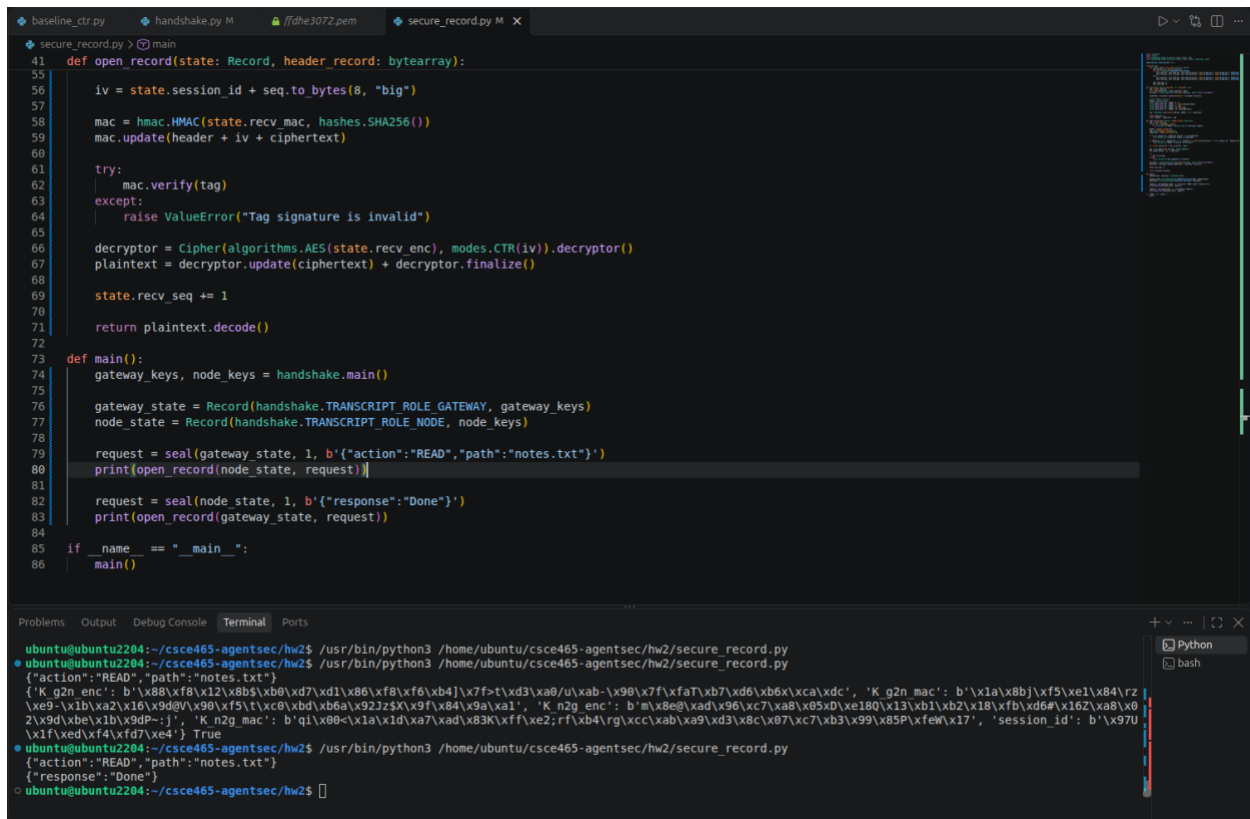
Because each signed transcript is comprised of both party identities, both ephemeral Diffie–Hellman public values and both random nonces, if an attacker were to modify their own Diffie–Hellman share the transcript would be changed and the signature would then fail. This gives us confidence in terms of integrity.

In the case of a reflection attack, the role tag being part of the signed transcript means that a signature made as the gateway for example cannot be verified as other parties as the signature won't match.

2.3 Without Signatures

If the signatures were to be removed, a MITM attack can be conducted such that an attacker runs a separate Diffie-Hellman exchange with each party during the handshake process. This will then allow for the attacker to read or change all traffic undetected as communication is unauthenticated.

Task 3: Encrypt-then-MAC Record Layer



```
41 def open_record(state: Record, header_record: bytearray):
55
56     iv = state.session_id + seq.to_bytes(8, "big")
57
58     mac = hmac.HMAC(state.recv_mac, hashes.SHA256())
59     mac.update(header + iv + ciphertext)
60
61     try:
62         mac.verify(tag)
63     except:
64         raise ValueError("Tag signature is invalid")
65
66     decryptor = Cipher(algorithms.AES(state.recv_enc), modes.CTR(iv)).decryptor()
67     plaintext = decryptor.update(ciphertext) + decryptor.finalize()
68
69     state.recv_seq += 1
70
71     return plaintext.decode()
72
73 def main():
74     gateway_keys, node_keys = handshake.main()
75
76     gateway_state = Record(handshake.TRANSCRIPT_ROLE_GATEWAY, gateway_keys)
77     node_state = Record(handshake.TRANSCRIPT_ROLE_NODE, node_keys)
78
79     request = seal(gateway_state, 1, b'{"action": "READ", "path": "notes.txt"}')
80     print(open_record(node_state, request))
81
82     request = seal(node_state, 1, b'{"response": "Done"}')
83     print(open_record(gateway_state, request))
84
85 if __name__ == "__main__":
86     main()
```

```
ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ /usr/bin/python3 /home/ubuntu/csce465-agentsec/hw2/secure_record.py
{"action": "READ", "path": "notes.txt"}
{'K.g2n.enc': b'\x88\xf8\x12\x8b\xbd\x7d\x86\xf8\xf6\xb4]\xf7>t\x03\xa0/u\xab-\x90\x7f\xfa\x7b\x66\xca\xdc', 'K.g2n.mac': b'\x1a\x8b\x5f\x5e\x18\x4r\x9e-\x1b\x02\x16\x9d@v\x90\xf5\t\x0b\xbd\x6a\x92jz\x5\x9f\x84\x9a\x01', 'K.n2g.enc': b'm\x8e@v\xad\x96\x07\x08\x05\x0d\x0e\x18\x13\x1b\x18\xfb\x06#\x16z\x08\x02\x9d\xbe\x1b\x9d~:}', 'K.n2g.mac': b'q\x00<\x1a\x1d\xa7\xad\x83K\xff\xe2;rf\x04rg\xcc\xab\xa9\x03\x0c\x07\x07\x03\x99\x85P\xfew\x17', 'session_id': b'\x97U\x1f\xed\x04\xfd7\xe4'} True
{"response": "Done"}
{'action': "READ", "path": "notes.txt"}
{"response": "Done"}
```

Figure 4. secure_record.py showing one record in each direction

Task 4: Adversarial Tests

```

tests > test_handshake.py > test_reflected_signature_rejected
29 def test_signature_over_different_transcript_rejected(transcript_hash):
34     handshake.verify_signature(sig, other_th, NODE)
35
36
37 def test_incorrect_rsa_public_key_rejected(transcript_hash, monkeypatch):
38     sig = handshake.sign_transcript(transcript_hash, NODE)
39     impostor = rsa.generate_private_key(public_exponent=65537, key_size=3072)
40     monkeypatch.setattr(handshake, "RSA_PUBLIC_KEY_NODE", impostor.public_key())
41     monkeypatch.setattr(handshake, "RSA_PUBLIC_KEY_GATEWAY", impostor.public_key())
42     with pytest.raises(ValueError, match="Peer signature could not be verified"):
43         handshake.verify_signature(sig, transcript_hash, NODE)
44
45
46 def test_reflected_signature_rejected(transcript_hash):
47     """The gateway's own signature, sent back as if it were the node's."""
48     gateway_sig = handshake.sign_transcript(transcript_hash, GATEWAY)
49     with pytest.raises(ValueError, match="Peer signature could not be verified"):
50         handshake.verify_signature(gateway_sig, transcript_hash, NODE)
51
52
53 def test_reflected_handshake_message_rejected():
54     """The gateway's first message bounced back to the gateway as the 'node reply'."""
55     gateway_msg = {"role": "Gateway", "identity": GATEWAY}
56     with pytest.raises(ValueError, match="Message role does not match expected"):
57         handshake.verify_message_role_and_iden(gateway_msg, "Node")
58
Problems Output Debug Console Terminal Ports
(.venv) juel@Juels-MacBook-Air-2 csce765_hw2 % python3 -m pytest -v
tests/test_records.py::test_sender_never_reuses_iv PASSED [ 58%]
tests/test_records.py::test_modified_ciphertext_rejected PASSED [ 61%]
tests/test_records.py::test_modified_tag_rejected PASSED [ 64%]
tests/test_records.py::test_ctr_bit_flip_attack_rejected PASSED [ 67%]
tests/test_records.py::test_modified_header_rejected[0-version] PASSED [ 70%]
tests/test_records.py::test_modified_header_rejected[1-direction] PASSED [ 74%]
tests/test_records.py::test_modified_header_rejected[9-sequence] PASSED [ 77%]
tests/test_records.py::test_modified_header_rejected[10-message_type] PASSED [ 80%]
tests/test_records.py::test_modified_length_field_rejected PASSED [ 83%]
tests/test_records.py::test_truncated_record_rejected PASSED [ 87%]
tests/test_records.py::test_replayed_record_rejected PASSED [ 90%]
tests/test_records.py::test_reordered_record_rejected PASSED [ 93%]
tests/test_records.py::test_reflected_record_rejected PASSED [ 96%]
tests/test_records.py::test_failed_record_does_not_desync_channel PASSED [100%]
===== 31 passed in 0.89s =====
(.venv) juel@Juels-MacBook-Air-2 csce765_hw2 %

```

Required test	Test function(s)	Asserted failure
1. Valid handshake & bidirectional messages	test_valid_handshake_derives_matching_keys, test_bidirectional_messages	keys equal & plaintext returned
2. Modified ciphertext	test_modified_ciphertext_rejected, test_ctr_bit_flip_attack_rejected	"Tag signature is invalid" error
3. Modified authenticated header	test_modified_header_rejected (version, direction, sequence, type), test_modified_length_field_rejected	"Header structure is incorrect" / "Ciphertext length is incorrect" errors
4. Replayed record	test_replayed_record_rejected, test_reordered_record_rejected	"Header structure is incorrect" error
5. Reflected record	test_reflected_record_rejected	"Header structure is incorrect"
6. Bad RSA key / signature / reflected handshake	test_incorrect_rsa_public_key_rejected, test_invalid_transcript_signature_rejected, test_reflected_signature_rejected, test_reflected_handshake_message_rejected	"Peer signature could not be verified" / "Message role does not match expected" errors

Security Note

Attacker Model and Non-Goals

For this assignment, I assumed our attacker is a man-in-the-middle attacker who can observe, modify, replay, reflect, and inject any handshake or record message. The attacker however does not hold either party's RSA private key and cannot break the underlying encryption. Against this attacker, this design aims to provide confidentiality, integrity, mutual authentication, replay and reflection resistance, as well as forward secrecy. Some non-goals include denial of service, traffic analysis of message sizes and timing, as well as judging whether an authenticated request is safe to execute. These are non-goals as they're more in line with security & authorization and not cryptography for this assignment.

Why CTR Requires Unique IVs

CTR mode operations require unique IVs as the encrypted counter output (the keystream block) is XORed with the plaintext block to produce the ciphertext. The keystream looks like this $C1 = P1 \oplus E1$. Since encryption algorithms are simply permutations, they are inherently deterministic. This deterministic nature means that for CTR mode operations, we will result in reusing the keystream if the IV is not unique as $C1 \oplus C2 = P1 \oplus P2$.

Why Verify-Before-Decrypt Matters

We want to always verify before because we never want to decrypt, parse, or act on data until we know it came from the legitimate sender unmodified. Until we have ran checks against the ciphertext to ensure it has not been tampered with, the ciphertext is unauthenticated and attacker controlled. Decrypting at this stage defeats the entire purpose of adding authentication. This is especially important for CTR mode operations as well as we previously discussed how flipping a bit in the ciphertext directly impacts the plaintext.

Why Independent Encryption and MAC Keys

The reason for these ties back to the concept of key separation. Having independent encryption and MAC keys means that a weakness or misuse in one of the schemes has a limited blast radius and doesn't compromise the entire system.

Comparison with AES-GCM

AES-GCM is an AEAD mode which provides much of the same guarantees as our construction in a single primitive. It combines CTR mode encryption with a built-in authenticator allowing for our header to be passed as associated data so it is authenticated without being encrypted. GCM uses one key instead of the two we implemented while still verifying the tag before returning any plaintext. The main drawback is that nonce reuse is far more dangerous in GCM as besides leaking the XOR of plaintexts, it also lets an attacker recover the authentication key and forge records. If the same were to occur with our current system, because the ciphertext is then hashed with HMAC, the system remains secure.

Valid Records Can Still Carry Dangerous Tool Calls

Although the record is valid, dangerous tool calls can still be carried as that's not the problem we are solving. Authentication & encryption are used to solve numerous problems such as proving who sent a message, proving it wasn't altered, as well as making it difficult to decipher the message unless you are

the intended recipient. Determining if the data / tool calls are dangerous or not is the job of authorization & policy checks. Without these, we can be assured that our data was not tampered with in flight but that doesn't stop a compromised gateway from sending compromised instructions to be executed.

AI Use

I utilized Claude to guide me with understanding the instructions and give me hints on how to continue when I was stuck. Almost all of the code is handwritten aside from some test cases & all code has been reviewed and verified by myself. Attached in the folder are both my AI_USAGE.md and AI_LOGS.txt files.